

Supplementary Material for Next: Enabling Continuous UTXO Payments with Direct Guarantees

Lu Hengrun

S1. DETAILS OF THE ORIGINAL-EXECUTION CONNECTION

Earliest conflicting polka. Fix a height and a commit certificate for a nonempty value v in round r , and select two honest signers K of that certificate. Each locks v before it issues its precommit. The certificate can be assembled after these signers have advanced, so its observation time is irrelevant.

Suppose a higher-round polka for a different value exists, counting nil if it permits unlocking. Select the smallest round $s > r$ with such a polka and, within s , the earliest event τ at which its distinct prevotes exist. Its three voters include a member $m \in K$. Honest rounds increase, so the round- s prevote of m follows its round- r precommit, and without an unlocking justification m must prevote its locked value. Relocking on v in intervening rounds preserves this restriction. A conflicting justification from a round below s contradicts the choice of s , and one from round s must have existed before m cast the vote counted at τ , which contradicts the choice of τ . Evidence from above s cannot precede a later round- s vote. No such polka exists, and a higher-round conflicting commit would require one. Same-round conflicting commits require an honest double precommit. The argument rests on the stated lock-transition and round-monotonicity rules and does not need timely receipt of commit evidence.

Pairing, validation, and execution. Induct on candidate updates. Local construction pairs a body with its original parts, and remote completion rebuilds the body from checked parts. Moving Locked or Valid transfers both, and switching targets clears the old body. Historical installation leaves these active candidates alone. A nonempty precommit follows a matching polka. An honest prevote has a justification in actual proposal validation or an earlier lock, and tracing the latter lowers the lock round until it reaches validation. The final full-identity guard fixes the original transactions, height, time, and other application context before Finalize. Synchronization and replay supply the same original material, so same-height retries cannot introduce a second original body through the stable-hash cache within this call boundary.

Write sets. The proofs of the main text rely on the following write sets. A compensation decision writes the issuer reserve account, the obligation status, the Promise and coverage record (Paid and Remaining), grant usage (Spent), the public gap, the consumer's payment record (Pending and fee stages), reward and refund balances and fee outputs, the removal of the due-index entry, and the decision record with its decision-index and pending-representation entries. The execution of a source after compensation writes the repayment: the issuer reserve account, the obligation status (Recovered), the coverage record (Recovered), and grant usage (Spent). It leaves the decision record unchanged. A resubmission job writes the member's local outbox only. A representation command or batch writes the logical revision, the head command, the task or batch task with its per-output references, the queue entry, and the removal of pending-representation entries. Installation writes local block-store versions and its own progress cursor only. No representation or installation write belongs to the economic projection.

Convergence for a fixed target set. Fix an original body and a set of decided slots. The replacement message of each slot is a function of the network and the output, so any packaging replaces the same slots by the same messages. If the chameleon relation has a unique valid opening for each fixed message and commitment, as holds for correctly generated parameters with a public exponent coprime to the group order, each slot opening and each part opening is the unique solution of its relation. Part contexts depend on the height and the part index and are independent of the revision counter. Any grouping or order of representation commands that covers the set therefore ends in the same body and part openings. Single-item commands, one batch, and batches of intermediate sizes differ in command identities, revision numbers, and application hashes. The statement concerns final bytes and depends on parameter generation and encoding as stated.

Follower updates. For a follower that starts from the same local state, configuration, and cursor, equal fixed payment fields and authenticated results induce equal economic updates whenever both representations decode. The

mutable funding reference and opening select no Paid, Pending, or fee effect. Decisions are authenticated in full, and the execution-result commitments authenticate their effects and fee outputs. Economic observation thus stays separate from the independent verification of the latest mutable historical body.

Relations used in the analysis. Three relations compare different objects. Representation inertness compares the economic projection before and after one representation or installation step, and it projects away command identities, revision counters, and materialization tasks. Fixed-target convergence compares the final body and part openings that different groupings and orders of representation commands produce for one original body and one decided set. Replay equality fixes the entire original command sequence and its block contexts, and deterministic execution then reproduces each application hash. Cross-history comparisons fix the economic commands, public times, and historical target coordinates, so projection equality does not identify entire application states and does not permit deleting or renumbering blocks. None of the three relations compares histories that order parent arrivals and compensations differently; the delay-neutrality theorem of the main text makes that comparison under its own hypotheses and equates final CAL holders and reserve balances, not application hashes or revision metadata.

Local invalidation write set and replay. Invalidation is evaluated only while applying an authenticated block entry whose execution result is successful and applied; `INSTALL` does not trigger it. Before the follower overwrites a spend record, it reads the previous candidate and verifies the immutable approval and the exact ordinary or user-fee input instance. Existing observation, installed certificate material, an outbox entry, settlement, pending obligations, compensation, recovery, or a fee effect is treated as an accounting contradiction rather than as releasable state.

The additional invalidation write set contains only the affected local resource slices and the approval’s local progress. For each debit it subtracts `Cap – Applied` from `Reserved`, adds the same amount to `Available`, sets the corresponding cumulative `Applied` entry to `Cap`, and records `Invalidated`, the first triggering public fact, and its height. The immutable approval and unrelated input locks remain. Overlay visibility makes later conflicting inputs or transactions in the same block no-ops for restoration; the follower cursor suppresses block replay, and the cumulative `Applied` vector prevents a second credit after restart. The member-store schema is advanced so that an older binary cannot silently ignore the invalidated terminal state. These writes commit with the successful consumption record and follower cursor. Schema 8 rejects earlier member stores; experiments use fresh directories rather than a migration.

A fully certified source that is permanently rejected by the global Intent binding while spending disjoint inputs is outside this invalidation rule: its QC-backed responsibility, input locks, and signing reservation remain. The current relay may continue retrying such a source; terminating those retries is a separate engineering follow-up and cannot release the certified liability.

S2. PER-RUN EVIDENCE FOR THE FROZEN EXPERIMENTAL BUILD

This section records frozen evaluation build 721800c, used by the main evaluation. Continuation and mixed-load trials share node binaries and Comet sources. The subsequent historical methods and archived tables retain their original build identities and are not throughput measurements of this build.

TABLE S1
FINAL-BUILD CONTINUATION; TIMES IN SECONDS FROM FIRST ACTUAL SEND.

Run	Last receipt	All public	Members closed	Attempts	Gaps
r1-fast	4.497998	5.228984	5.293857	100	3
r1-wait	64.626603	65.232083	65.284395	149	0
r2-fast	4.739972	5.253720	5.295722	100	2
r2-wait	64.475560	65.105699	65.154442	116	0
r3-fast	4.556805	5.114055	5.129723	100	4
r3-wait	64.460699	65.075768	65.124437	142	0

These costs come from offline successful-execution fixtures. Representation work includes sequential adaptation, assembly, Finalize, synchronous application Commit, and materialization; common compensation decisions are reported separately. Network and consensus waits, public-block saving, and commit-signature construction are

TABLE S2

ORDINARY PAYMENTS PER MIXED-LOAD RUN; LATENCIES IN MS, SPAN AND DRAIN IN S. FINAL COLUMNS COUNT OUTSTANDING-LIMIT AND FAST-CONCURRENCY-LIMIT HITS.

Run	Receipt P50	Receipt P95	Dispatch P95	Span	Drain	Total cap	Fast cap
r1-C	53.435	136.221	201.339	70.191	1.029	1	0
r1-N	55.259	152.766	255.396	70.245	1.015	16	0
r1-P	55.102	154.780	895.879	70.886	1.356	63	935
r1-R	53.034	139.894	0.239	69.990	0.815	0	0
r2-C	54.011	136.490	700.329	70.690	0.906	68	760
r2-N	53.653	143.869	110.389	70.416	0.820	18	0
r2-P	52.892	134.963	0.242	69.990	1.066	0	0
r2-R	53.832	144.903	0.218	69.990	1.125	0	0
r3-C	55.682	159.437	405.296	70.395	1.046	65	2
r3-N	55.132	155.992	0.242	69.977	1.200	5	0
r3-P	54.996	144.206	586.541	70.576	1.155	81	549
r3-R	52.706	126.355	0.186	69.984	0.890	0	0

TABLE S3

HISTORICAL REPRESENTATION PER PROCESS; MAINTENANCE AND COMMON DECISION EXECUTION/COMMIT IN MS, NET ADDITIONAL KV IN BYTES.

Process	Payments	Compensated	Representation	Net extra KV	Common decision
1	32	1	43.322	292136	8.926
2	32	1	43.664	292200	8.917
3	32	1	42.035	292162	9.013
1	32	8	97.183	329118	11.919
2	32	8	94.461	329126	11.984
3	32	8	93.591	329059	12.029
1	256	1	64.826	2258078	9.052
2	256	1	66.931	2258044	9.989
3	256	1	65.323	2258085	8.993
1	256	8	214.398	2294947	20.119
2	256	8	213.902	2294910	21.027
3	256	8	214.781	2294980	20.967

outside this timer. KV is the logical increase in encoded records, not the physical disk-space difference between two deployments.

Subsequent reclamation regression. Build `bf71c4d` adds the superseded-partial-approval transition. Directed tests cover exact ordinary and fee-input conflicts, duplicate processing, restart, refusal to re-sign, and the disjoint-input Intent counterexample. A separate fresh-start nine-service run with synchronous member storage completes 1,000 user-funded payments offered at 100 payments/s (9.990-s send span, 10.879-s total), and a 100-hop fast chain reaches its last certified receipt in 4.422 s and closes in 5.014 s. Both runs have matching committee state and empty outboxes. These correctness regressions do not replace the performance measurements of `721800c`.

S3. EXPERIMENTAL METHODS FOR ARCHIVED BUILDS

Build lineage and deployment. Results retain their original build identifiers (Table S4). The *continuation and accounting baseline*, committed as `d99ec92`, implements the layered continuation and the accounting transitions; its compensation commits together with its representation. The artifact records per-binary SHA-256 fingerprints and the configuration of each run. The nine-service continuation, compensation and repayment, storage, and short-throughput series use this build. The initial fourteen-service paired runs use the same single-obligation rule before the batch-result tag was separated from the direct-result tag, and the cross-type codec regressions cover that separation.

TABLE S4
BUILDS AND THE SERIES THEY SUPPORT.

Build	Identifier	Series
Continuation and accounting baseline	d99ec92	100-hop chains, synchronous-member and storage controls, compensation and repayment matrix, paired interference runs, short throughput.
Decision-first	Base 0e7bae5; functional package 2c91b0251024322c; later series snapshot 449b4d4072c99588; audited f185f73	Nine functional runs, six ordinary-path runs, regression and race tests.
Frozen evidence	main-text 721800c	Final 100-hop chains, mixed traffic, and historical-reading controls.
Partial-approval reclamation	bf71c4d	Conflict and restart regressions; 1,000-payment smoke and 100-hop continuation.
Archived	e3870e1/a083ff6, f0c0a274, 8e1fb40/5807546, bb65af69/13be856, f82f8cd, c1ebf2d	Operating range, original continuation, capital, organization paths, batch integration, source-failure checkpoint.

Members in the baseline storage, in-memory continuation, and short-throughput controls run with GOGC=200 and GOMAXPROCS=16; this override is not applied to committee, gateway, or wallet processes.

The *decision-first build* (accounting domain `DIRECT_ACCOUNTING_V5_DECISION_FIRST`) implements re-submission from public certificates, the compensation decision, and the separate representation commands. It derives from the branch base 0e7bae5, a documentation-only descendant of d99ec92. The nine functional runs use the initial implementation snapshot, whose source-package SHA-256 begins 2c91b0251024322c. The later snapshot of this series, whose package SHA-256 begins 449b4d4072c99588, rejects a decision whose position does not match committed state before reading any historical block and raises the bounded due-decision scan from 8 to 32 entries per round; the ordinary payment path is unchanged. The six ordinary-path runs, the full test suite, and the race tests use the later snapshot of this series. The audited commit f185f73 additionally requires the certificates attached to a payment to correspond one-to-one to its certificate inputs, and it does not change the payments accepted in the measured workloads. Numbers from the baseline and decision-first builds are reported separately.

The archived operating-range runs record base e3870e1 and implementation a083ff6. Original continuation records f0c0a274, capital records base 8e1fb40 and implementation 5807546, and organization paths record base bb65af69 and verified source 13be856. The archived batch integration uses f82f8cd, and the structural repair counts were repeated on the baseline code. The source-failure checkpoint c1ebf2d uses the superseded late-source rule and does not support the current claims.

Mac trials share one M4 Max host with 16 CPU cores and 64 GiB RAM, and the structural microbenchmarks run on Windows amd64. Baseline memory chains and short throughput trials use in-memory service state and NoSync wallets. Compensation and repayment trials use disk-backed committee application state and BlockStore with in-memory members and gateway. Storage controls change only member commits between bbolt NoSync and synchronous transactions. Each run starts from fresh genesis. Six additional fourteen-service disk-backed chain runs use the baseline accounting rule and user-paid FUEL. Their three fast 100-hop runs take 668.960–688.080 ms, and the inter-hop confirmation runs take 114.496–118.742 s. They form a separate storage and deployment configuration from the nine-service results in the main paper.

Decision-first functional runs. The nine runs use the fourteen-service configuration with 450 initial outputs, a disk-backed committee, members in their default NoSync mode, flush and gossip settings of 10 ms, and explicitly authorized organization-funded FUEL; signatures and all amount and consumption checks remain enabled. Ordinary-payment stress wallets use the NoSync wallet option, and the two-hop fault demonstrations use the default wallet mode. In each run, an injected fault prevents the original publisher from submitting the parent payment or running INSTALL for it after its full QC exists, so the original signers hold their approvals but not the QC. The child spends the parent’s TXCer. In the resubmission treatment the driver never submits the original payment. The

TABLE S5

PER-RUN RECORDS OF THE NINE DECISION-FIRST FUNCTIONAL RUNS. TIMES ARE CLIENT OBSERVATIONS FROM THE CHILD PAYMENT'S WALLET SEND; "PARENT FAST" IS THE ORIGINAL PAYMENT'S OWN RECEIPT. PUBLIC COLUMNS INCLUDE POLLING. A DASH MARKS A COLUMN THAT DOES NOT APPLY OR WHOSE TIME WAS NOT RECORDED. "AUTO. SOURCE" DOES NOT APPLY IN THE CONTROL AND PAUSED ROWS BECAUSE AUTOMATIC RESUBMISSION IS DISABLED THERE; THE DRIVER LATER SUBMITS THE SOURCE, WHICH EXECUTES AND REPAYS THE ISSUER (VERIFIED BY THE REPAYMENT AUDIT), BUT THAT INSTANT WAS NOT TIMESTAMPED. IN THE RESUBMISSION ROWS NO COMPENSATION OR MATERIALIZATION OCCURS.

Treatment	Run	Child fast (ms)	Parent fast (ms)	Child public (s)	Auto. source (s)	Compensation (s)	Materialized (s)
Resubmission	1	13.94	9.02	1.414	2.014	–	–
	2	13.94	8.94	1.990	2.565	–	–
	3	13.91	9.02	1.440	2.039	–	–
Control	1	11.96	8.97	1.364	–	35.839	36.994
	2	12.94	8.97	1.413	–	35.888	37.021
	3	12.01	9.00	1.388	–	35.864	37.019
Paused	1	12.00	7.98	1.362	–	35.837	42.748
	2	13.92	9.04	1.341	–	35.814	42.736
	3	13.92	9.03	1.841	–	35.891	42.798

control treatment disables the automatic resubmission step with an experiment switch. Ordinary backup submission by members holding installed complete material remains enabled; INSTALL itself replicates that material within the organization. The paused treatment also disables resubmission and, while a marker file exists, makes the input and part adaptation HTTP endpoints of all four committee members return a paused status; consensus, decisions, and payment processing are not paused.

Times start at the first HTTP send of the child payment. The fast receipt ends when the wallet has verified the certificate and recorded the output. Public events are observed by polling the followers' status endpoints, so they include polling delay, and the difference between two public observations is not a protocol delay measured from the exact commit. The compensation time includes the configured 30-s obligation period, the advance of public time by committee ticks, and background scheduling. In the paused treatment the script submits the late source after the compensation and waits until the status endpoints of all four committee members show the obligation repaid and the representation neither committed nor materialized. It stores these four observations, removes the marker, and then waits for the representation to be committed and materialized at all four members. The status endpoint's stability flag compares the header hash, data hash, and transaction identity. The materialization step and the real-consensus integration tests check the complete block identity and part-set header, so the flag alone is not offered as the complete identity proof. The repayment and the resumption of adaptation have no timestamps of their own. The measurements contain no comparison with periodic scanning, vote collection, or the earlier coupled build under paused adaptation.

Each treatment has three rounds, and every functional run passes four checks: equal state hashes at the four committee members, a zero gap, closed payments, and drained outboxes. The resubmission and control treatments each carry three additional runs of 300 ordinary payments. Per-run values are in Table S5.

Decision-first ordinary-path runs. Six runs compare the build at the branch base with the final decision-first snapshot in the order AB, BA, AB. Each sends 300 payments at 100 payments/s with 64 concurrent fast requests and at most 256 outstanding, using NoSync stress wallets. The elapsed time includes the public confirmation of all 300 payments and member following; it is neither a single-payment time nor a maximum throughput. The send span is about 2.990s in every run. Summary values in the main text are medians of the per-run statistics, not quantiles pooled over payments. Ordinary final outputs carry no input certificates, so these runs do not measure the storage retained for payments whose inputs are not final. Per-run values are in Table S6.

Regression coverage. Compensation decisions are rejected when they are not yet due, when the network, slot, or amount does not match, when the reserve is insufficient, or when they conflict with the recorded decision. A repeated decision has no economic effect, including after repayment, where it returns no changes, effects, or fee outputs, and both orders, source first and decision first, are covered. Resubmission is tested for cross-organization

TABLE S6

PER-RUN RECORDS OF THE SIX ORDINARY-PATH RUNS. RUN ORDERS ARE AB, BA, AB (A = BRANCH BASE, B = DECISION-FIRST). QUANTILES ARE OVER THE 300 PAYMENTS OF ONE RUN.

Build	Run	Elapsed (s)	Fast P50 (ms)	Fast P95 (ms)	Block P50 (ms)	Dispatch P95 (ms)	Send span (s)
Branch base	1	3.871	1.112	27.441	1151.0	0.338	2.990
	2	3.815	1.096	17.951	1101.1	0.295	2.990
	3	3.805	1.052	39.945	1076.0	0.437	2.990
Decision-first	1	4.000	1.071	30.530	1127.9	0.430	2.990
	2	3.845	1.073	22.084	1103.9	0.355	2.990
	3	3.791	1.045	33.744	1052.3	0.383	2.990

and multi-level sources, repeated exposure, and payments already observed or already queued, which are not rebuilt. Ordinary verification rejects mismatched material, and the original approval, the payment bytes, and the CAL quota are unchanged before and after resubmission. Representation requires a successful decision, remains possible after the source’s repayment, and does not restore member progress or Worker slices a second time. For identity and replay, real threshold adaptation preserves the complete consensus identity, the successor body, and pre-signed successor references. Replay of the original history reproduces the application state at every height, and item-wise and differently grouped representation of one decided set give the same final body and part openings, with 1, 2, 4, and 8 inputs spanning two real parts. The full test suite and static checks pass at the later snapshot of this series on Windows and macOS, the race detector passes on the member, representation, and committee-runtime packages on macOS, and the full suite and static checks pass again at the audited commit.

Timing and aggregation. Certified-receipt latency generally runs from the first wallet HTTP send through recipient verification and local atomic recording. Capital and source-failure drivers include a small amount of pre-send construction, so their comparisons stay within the same driver family. A local NoSync transaction completion is not a synchronous durability receipt. Public and member completion timestamps record observation of authenticated execution, not the committee’s internal commit. The sender’s single monotonic clock measures cross-site ACK latency, including the return path, and a recipient can start its successor before the preceding sender receives that ACK. Offered targets, actual send rates, and completion rates including drain are retained separately. Captions distinguish transaction quantiles from statistics across runs, and within-run sample variability is not an estimate of cross-run uncertainty.

Delivery-gate runs. On the archived delivery-gate build, the gated variant waits for three members to INSTALL complete material or for authenticated public success, whichever occurs first, and collects no second signature quorum. At 200 payments/s for 30 s, three paired runs per mode each send and complete 6,000 payments. The median of run-level receipt P50s is 1.300 ms with immediate delivery and 2.167 ms with the gate. Separate 100-hop trials reduce median chain time from 287.983 to 200.356 ms (Fig. S1).

Baseline-build controls. The reserve trajectories of the three 5% runs of the compensation and repayment matrix (Fig. S2) are sampled at the driver’s release schedule, so the lowest sampled reserve is not a reserve-sizing result.

Three pairs of fourteen-service runs, ordered normal/withheld, withheld/normal, and normal/withheld, use the initial baseline build, before the batch-result tag separation, with committee and member databases on disk and member and wallet NoSync. Each sends 4,000 payments at 100 payments/s for 40 s. Alongside them, four additional two-hop demonstrations have sources that are delivered normally (normal run) or withheld, compensated after 30 s, and later repaid (withheld run). All 4,000 payments complete in all six runs. The final CAL held by each owner role (30,000 and 830,000) and both reserve balances (10^{12}) coincide across the six runs, and the withheld runs record 400 CAL paid and 400 CAL repaid, so the final allocation is consistent with delay neutrality. Receipt P50s for normal and withheld runs are 1.070/1.127, 1.089/1.108, and 1.083/1.097 ms, and P95s are 26.84/59.23, 29.49/58.44, and 27.41/55.15 ms; completion rates are 97.2–97.8 payments/s in all runs. Receipt P50 is 1.3–5.3% higher and P95 is 1.98–2.21 times the paired normal values. Concurrent compensation therefore affects the receipt tail in this configuration, and these pairs do not isolate the responsible stage.

Six runs of 1,000 payments at 100 payments/s on the nine-service configuration (committee on disk, wallets

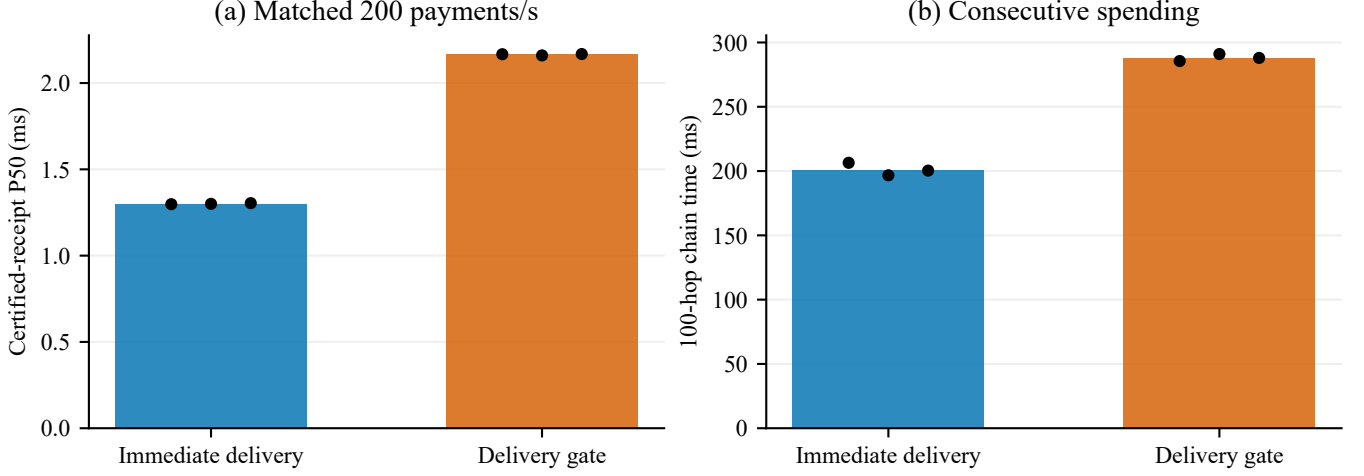


Fig. S1. Delivery-gate effect (ms). Left: run-level receipt P50 at 200 payments/s, 6,000 payments/run, three paired repetitions. Right: whole-chain time for three 100-hop chains per mode. Columns show the median of the three run values; dots show individual runs.

NoSync) differ only in the member store. With NoSync members, receipt P50 is 1.002–1.035 ms and P95 33.97–40.62 ms, with 93.11–93.93 payments/s completed including drain. With synchronous member commits, P50 is 43.19–51.85 ms and P95 80.98–96.56 ms, with 90.99–92.72 payments/s. All payments complete. Because wallets remain NoSync, this isolates the cost of synchronous member commits.

Representation microbenchmarks and batch integration. These structural measurements use the earlier coupled representation command, which also carried the economic step. Microbenchmarks on Windows amd64 with Go 1.27.1 place $k \in \{1, 2, 4, 8\}$ missing inputs in one historical block and part. Across five repeats, single commands require k commands and k part adaptations, whereas a batch uses one of each, and input adaptations remain k . At $k = 8$, total encoded command bytes fall from 127,336 to 2,752 (Fig. S3). The dispersed control separates batching from compact encoding: eight independent target blocks still require eight commands and eight part adaptations, while bytes fall from 30,008 to 5,440. At $k = 1$, both formats perform one part adaptation and encode 3,751 versus 680 bytes. Compact fields therefore explain part of the byte reduction, and shared block representation explains the reduced adaptation count. These are encoded command sizes, not total network traffic or total cryptographic work. They also exclude the 88-byte decision command that each decided output carries in the decision-first build.

An archived fourteen-service, disk-backed batch-integration run (`f82f8cd`) combines four actual missing sources into a single 1,568-byte batch whose BatchID agrees across all committee replicas. After late-parent handling, all 1,000 subsequent payments offered at 100 payments/s complete. Receipt P50/P95 are 1.048/26.531 ms, and total completion time is 11.120 s, including observation and shutdown. Gaps and outboxes drain, and committee state hashes agree. This batch carried economic steps and predates the separation of decisions from representation.

Externally funded authorization controller. The capital experiments sample member availability every 200 ms. Let ρ be the declared CAL authorization demand per second, counting both parent and child requests in each workload unit, and let u count unique requests whose certification remains unfinished. The controller sets

$$\begin{aligned} b &= 200 + 100u, \\ H_{\text{low}} &= \rho(0.8\text{ s}) + b, \\ H_{\text{target}} &= 1.25H_{\text{low}}. \end{aligned}$$

Here the watermarks and burst allowance are in CAL. When the minimum observed member availability A_{min} falls below H_{low} , the proposed grant increment is

$$\Delta B = 100 \left\lceil \frac{1.5(H_{\text{target}} - A_{\text{min}})}{100} \right\rceil,$$

subject to the configured authorization cap and available external funding. At most one increment is in flight. Its public command debits the external source and credits the reserve and grant atomically, and following members

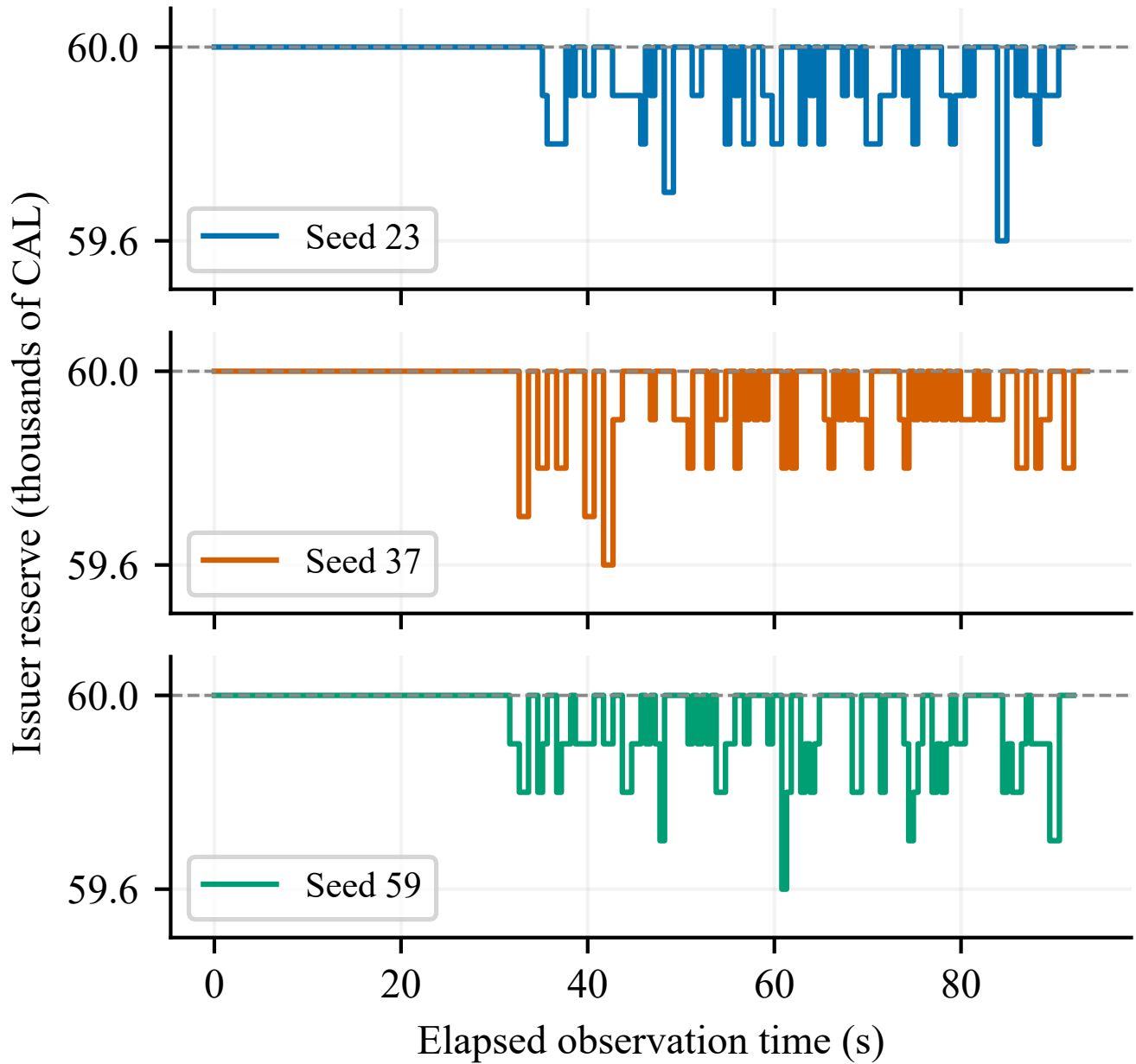


Fig. S2. Sampled reserve trajectories for the three 5% runs (60 compensated sources each). Repayment restores the initial 60,000 CAL. The vertical axis is restricted to 59,600–60,000 CAL to show temporary use. Samples reflect the driver's release schedule.

add the resulting share difference without clearing existing approvals. The experiment supplies ρ in advance rather than estimating unknown production demand. Controller decisions are requests for actual funded transfers, not a mechanism for creating backing capital.

Capital workload accounting. Each condition has one 300-s run with one Worker per member and sufficient user FUEL. Starting from 3,600 CAL, the constant, rate-step, and delayed-parent conditions end at 14,400, 21,600, and 23,300 CAL after external transfers of 10,800, 18,000, and 19,700 CAL (Table S7). Each completes 6,000 two-hop units, or 12,000 payments, spending 1,128,000 user FUEL. Corresponding fixed-high controls complete, while the fixed-low condition closes 46 of 6,000 units, leaves 5,826 unstarted, and records 64 partially approved payments. These categories have different units and are not summed as one payment count. The per-run tables preserve the full seven-condition comparison without implying three repetitions per condition. The series ran on an earlier build in which no compensation occurred.

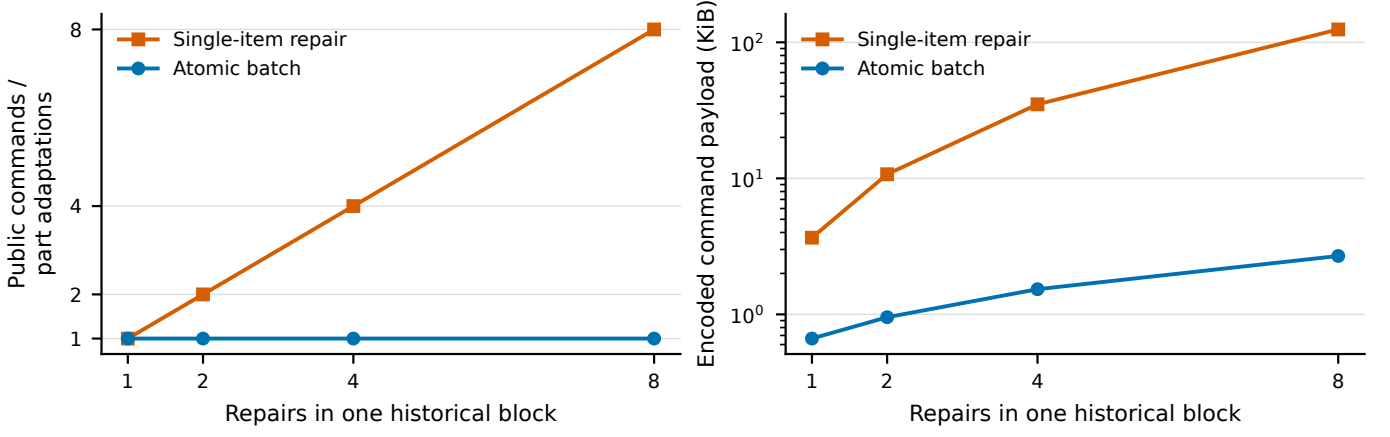


Fig. S3. Same-block, same-part representation costs of the earlier coupled command versus obligation count k . Panels show command/part-adaptation counts and total encoded command payloads in KiB (log scale). Five repeats per configuration give identical structural measurements. Input adaptations remain k .

TABLE S7

ACTUAL CAPITAL ADDITIONS. EACH CONDITION HAS ONE 300-S RUN, STARTS WITH 3,600 CAL, AND COMPLETES 12,000 PAYMENTS. BOTH COLUMNS ARE CAL; EVERY ADDITION DEBITS AN EXTERNAL FUNDED ACCOUNT.

Workload	Final reserve	Added capital
Constant: 20 units/s	14,400	10,800
Step: 10 to 30 units/s	21,600	18,000
Parent delayed by 3 s	23,300	19,700

Identities, resource use, and retained history. The multi-identity configurations use shared clients, so 2,048 wallet identities do not represent 2,048 separate operating system processes. Comparing independent inputs with ten-hop payments, the three-run medians are 12.67 versus 15.24 core-ms per payment and 59.32 versus 71.83 KiB of HTTP payload per payment. Memory reporting separates total node RSS and retained history from active obligations, pending work, and outboxes, so an increasing historical footprint is not reported as an increasing payment backlog. Per-run resource summaries and their archived source locations are included in the artifact index.

Endpoint-specific Lightning reference. The archived deployment uses LND v0.21.3-beta and Bitcoin Core 31.1 in regtest, with synchronous LND persistence and its 50-ms channel-commit setting. Each of three fresh deployments has two nodes and a direct channel. After 20 warm-up transfers, each run issues 100 serial, alternating 10,000-satoshi payments using prepared invoices, with receiver `SETTLED` as the completion event; the reported P50/P95 are 282.12/320.17 ms. A separate Next run sends 300 payments at 100 payments/s for 3 s using in-memory service state and NoSync wallet storage. Recipient-certified receipt has P50/P95 of 0.9625/1.2075 ms. Workload concurrency, durability, and completion semantics differ, so these records are deployment references rather than a normalized acceleration ratio.

A. Archived Operating-Range Series

The archived operating-range series complements the continuation results. One active organization uses sponsored fees, reused addresses, and independent final inputs. Five runs of 150,000 payments each all finish (Fig. S4). At a target of 2,000 payments/s, two runs achieve completion rates of 1,948.66–1,950.45 payments/s, including drain, with receipt P50s of 2.521–2.566 ms. Raising the target to 2,200 yields 2,103.63–2,106.29 completed payments/s. At 2,400, the achieved rate is 2,174.68, P50/P95 rise to 7.174/89.38 ms, and the 4,096-entry pending admission limit records 1,635 hits. The operating curve pairs achieved completion rates with their corresponding queue pressure.

S4. RESULTS FROM EARLIER BUILDS

These results retain their original builds, storage modes, and measurement endpoints. The main paper reports the three core experiments on frozen evaluation build 721800c. Earlier performance and CPU measurements use

TABLE S8

COMPENSATION DECISION RECORD AND REPRESENTATION COMMANDS (C3). BOTH ARE PUBLIC COMMANDS COMMITTED ONCE; THEY DIFFER IN WHAT THE STORED HISTORY SHOWS.

	Decision record	Decision plus representation (C3)
Economic closure	Reserve debit, obligation state, and fee effects commit in one public decision, independent of adaptation; successors unchanged.	The same decision; representation commands add no economic effect.
Stored historical body	Original body unchanged; the funding source of a consumed input is read from the decision index.	Stored body names the reserve debit; transaction identity, owner authorization, and complete block identity (H_b, P_b) are unchanged and verified.
Inspecting a designated historical input	Consult the decision index together with the block.	Read the represented source under the original commitments and consult the permanent decision and current obligation state in the same V_H .
Readers	An honest local replica that has verified and executed prefix H .	The same local replica, reading a committed logical revision in V_H ; remote untrusted readers need separate authorization evidence.
Storage and replay	Original blocks plus decision records; replay runs the original commands, including decisions.	Additionally revised bodies, revisions, and tasks; replay runs the same original commands and the representation commands; same-block commands share part adaptation and installation advances to the latest committed revision.
Progress dependency	Public execution of the decision.	Three adaptation contributions and a current revision; economic closure does not depend on them.
Status	Implemented as the economic path; measured in the decision-first runs.	Implemented; functional runs and regressions above.

different builds and workloads and are not combined into a capacity estimate for one configuration.

A. Continuous Spending Without Per-Hop Settlement

We compare certified continuation with waiting for public execution between hops. On the baseline build, a 100-hop chain reaches its last certified receipt in a median 158.846 ms, compared with 65.204 s when each hop waits for public confirmation. Three wallets transfer 100 CAL in rotation, constructing each payment online from its predecessor’s output and paying fees from their own final FUEL. Three repetitions per mode give six runs and 600 payments on the nine-service in-memory configuration. Both modes end at the last certified receipt, and the waiting mode adds a public confirmation *between* hops. With committee commit, flush, and gossip settings of 250, 10, and 10 ms, public execution and its observation cost about 0.65 s per hop. Certified continuation removes that wait from the recipient’s path.

Figure S5 shows the six whole-chain durations. The fast runs take 158.492–159.905 ms. All 297 fast-mode successor requests consume the preceding certified output and are sent after its certified receipt but before its public completion is observed. A one-input, one-output TXCer is 779 bytes, and requests are 1,979 bytes at the first hop and 2,766 bytes thereafter. For this transaction shape, the direct evidence stays the same size across all 100 hops, because each request carries the certificates of its direct inputs rather than the ancestor chain.

To test whether the advantage depends on in-memory member storage, three paired 100-hop trials on a nine-service disk-backed configuration with NoSync wallets and user-paid FUEL differ only in member storage (NoSync or synchronous; order N/S, S/N, N/S), and each is followed by a synchronous waiting chain. Median durations are 681.127 ms for NoSync continuation, 4.471 s for synchronous continuation, and 65.013 s for synchronous waiting. Certified continuation therefore remains 14.5 times faster than waiting when member commits are synchronous. All 900 payments complete with agreeing committee states.

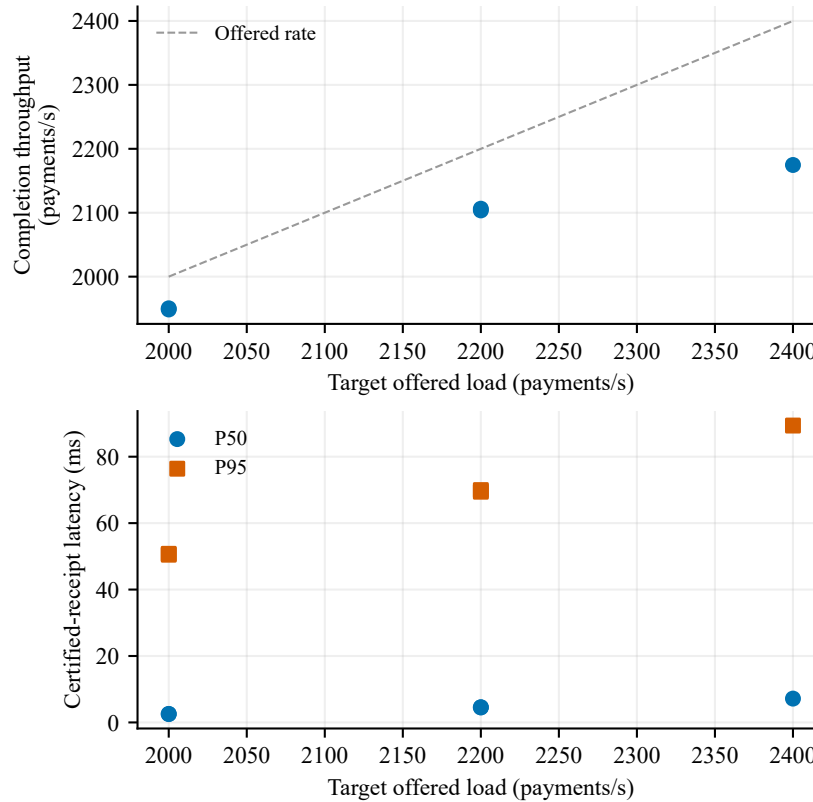


Fig. S4. Operating points versus target offered load. Top: completion throughput (payments/s), including drain. Bottom: receipt latency (ms). Each point is one 150,000-payment run: two at target 2,000, two at 2,200, and one at 2,400. P50/P95 summarize transactions within each run, not cross-run uncertainty. All services share one host.

On an archived delivery-gate build, a gate after certification waits for three members to `INSTALL` complete material or for authenticated public success. The gate collects no second signature quorum, and delivering immediately instead shortens continuation. At 200 payments/s for 30 s, three paired runs per mode each complete 6,000 payments, and the median of run-level receipt P50s falls from 2.167 ms with the gate to 1.300 ms (40.0%). Separate 100-hop trials fall from 287.983 to 200.356 ms (30.43%). Authorization is unchanged, and the supplement lists the paired runs.

B. Funded Closure and Repayment

This experiment checks that compensation leaves successor payments in place and that each compensation is repaid exactly once when its source later executes.

Compensation and repayment matrix (baseline build). Nine runs use seeds 23, 37, and 59 at withheld-parent rates of 0%, 1%, and 5%. Each run offers 20 two-hop units/s for 60 s (1,200 units, 2,400 payments) against a fixed 60,000-CAL reserve with user-paid FUEL and no top-up. The nine-service deployment uses disk-backed committee application state and BlockStore, in-memory members and gateway, and NoSync wallets. The driver releases each withheld parent only after its compensation command is committed and its historical revision is materialized on all four replicas, so every withheld source is compensated after the 30-s public-time deadline and executes afterwards.

All 21,600 payments close. The 216 compensations (12 per 1% run, 60 per 5% run) are each followed by repayment. At the end of every run no obligation is Open or Repaired, the gap is zero, net Spent and Reserved are zero, and the reserve is back at 60,000 CAL (Table S9). The lowest sampled reserve is 59,800–59,900 CAL at 1% and 59,600 CAL at 5%. It reflects the driver’s release schedule rather than a reserve-sizing result, and the supplement plots the three 5% trajectories. Run-level receipt P50s for normal child payments are 1.185–1.251 ms. Measured from the deadline, the committee commit, which in this build carries the representation, is observed at

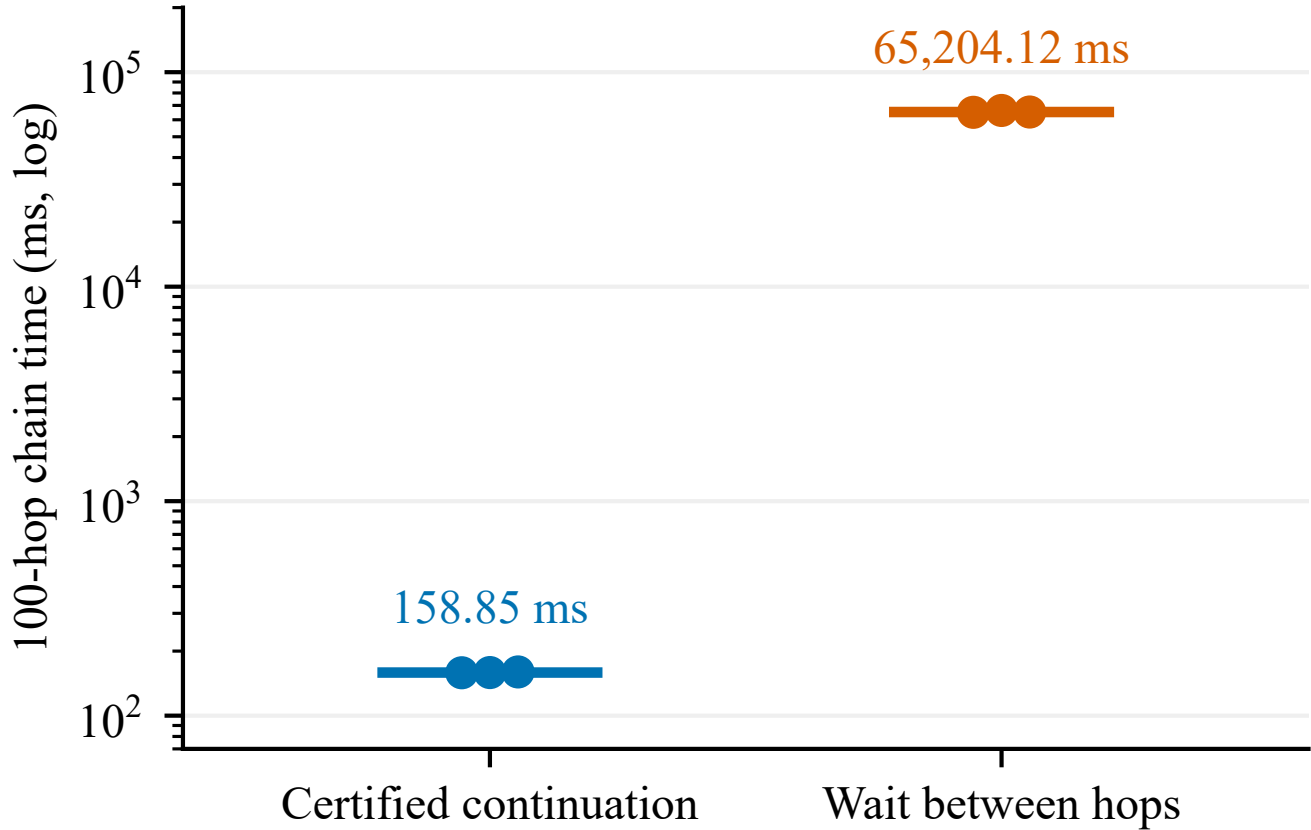


Fig. S5. Time through the last certified receipt of a 100-hop chain on the baseline build. Dots are three independent runs per mode; horizontal bars mark their medians. The logarithmic time axis includes the same endpoint in both modes. Only the waiting baseline requires public confirmation between successive payments.

TABLE S9

CLOSING AUDIT OF THE COMPENSATION AND REPAYMENT MATRIX. EACH COLUMN IS REPRODUCED BY ALL THREE SEEDS; EACH RUN HAS 2,400 PAYMENTS. RESERVE VALUES ARE CAL, FEE VALUES ARE IN THOUSANDS OF FUEL, AND FEE CAPS SUM THE PER-PAYMENT MAXIMA.

Withheld parents	0%	1%	5%
Compensations / repayments	0	12	60
Paid / repaid CAL	0	1,200	6,000
Final reserve	60,000	60,000	60,000
Unclosed / net Spent	0	0	0
Total fee caps	2,400	2,400	2,400
Rewards	201.600	201.660	201.900
Burned	24.000	24.000	24.000
Refunded	2,174.400	2,174.340	2,174.100
Remaining Held	0	0	0

a median of 1.003–1.254 s, and physical installation of the revised body on all four replicas at a median of 1.257–1.761 s (P95 up to 2.015 s); both include 250-ms observation polling. Each normal payment spends 94 FUEL and each compensation adds 5, and rewards, burn, and refunds sum to the fee caps with zero Held.

Ledger-level regression covers arrival orders directly: all 24 arrival orders of a four-hop chain, with and without due compensation, under three issuer sequences give 144 trajectories, and 48 split–merge trajectories vary all arrival orders of a 40/60 split, two branches, and their merge under two compensation modes. Every step in the

192 trajectories conserves the independently computed asset projection. Every trajectory ends with each issuer's reserve restored, Spent and Reserved zero, Paid = Rec, and the consumed inputs still consumed.

Two fourteen-service controls separate permanent loss from delayed repayment. When the parent never arrives, the grandchild keeps its 100 CAL and the responsible issuer's reserve stays 100 CAL lower. When a cross-organization chain arrives grandchild first, the intermediate source closes its own direct obligation, and the root issuer's compensated 100 CAL is repaid when the root later executes. The four committee states agree in both cases, and an absent parent's own fee and work reservations can remain outstanding.

Three paired fourteen-service runs on the initial baseline build, before the batch-result tag separation (4,000 payments at 100 payments/s each), end with identical owner holdings and reserve balances in normal and withheld-source runs, consistent with the CAL delay-neutrality theorem in the main paper. Concurrent compensation raises receipt P95 by a factor of 1.98–2.21, and the pairs do not isolate the responsible stage.

A storage control with NoSync wallets isolates member commits: receipt P50 is 1.0 ms with NoSync members and 43–52 ms with synchronous commits.

An earlier-build capital series with one run per condition and no compensation shows that external additions of 10,800–19,700 CAL sustain 300-s runs of 6,000 two-hop units from a 3,600-CAL start, whereas a fixed 3,600-CAL control closes 46 of 6,000 units.

C. Continuation Across Registered Organizations

To measure the effect of communication distance, we use a two-organization suite on an archived build recorded in the supplement. Its 24 runs complete 342,000 payments, with two wallet processes holding 32 identities each across 64 lanes and each organization supplying 50 payments/s. The latency comparison uses ten-hop same- and cross-organization workloads at 0, 20, and 100 ms injected HTTP RTT, three repetitions per condition. Each proxy direction adds half the RTT, and intra-organization certification and committee P2P receive no delay injection.

For the common first-120-s cohort, the median of three run-level cross-organization ACK P50s is 1.632, 21.658, and 101.967 ms, respectively, and same-organization values span 1.615–1.744 ms (Fig. S6). Cross-organization ACK P50 tracks the injected RTT, with a measured excess of 1.6–2.0 ms. An ACK includes the return path, but a recipient may start its successor on acceptance without waiting for the ACK to reach the previous sender. At 100-ms injected RTT in the cross-organization workload, three 300-s runs each complete 30,000 payments; pending peaks are 106–110 and drain without sustained tail growth. The experiment isolates HTTP path delay within one physical host.

D. Operating Capacity and Service Availability

Independent-input throughput (baseline build). Three 30,000-payment runs target 2,000 payments/s on the nine-service in-memory configuration with NoSync wallets and authorized organization-paid FUEL. Member processes use a 200% Go GC target and 16 runtime processors. All payments complete and the four committee states agree. Completion rates including drain are 1,750.27, 1,820.91, and 1,887.31 payments/s over 15.9–17.1 s, with receipt P50s of 2.646, 2.842, and 2.824 ms and P95s of 47.9, 49.1, and 48.0 ms. The driver recorded 6,551–7,570 payments that had to wait for a fast-receipt admission slot, and the P95 dispatch lag was 179–229 ms, so the offered load was not uniform at 2,000 payments/s. These are short-run completion measurements on the baseline build. They are not remeasured for the decision-first build, whose cost on ordinary payments is tested by the six-run comparison above. The supplement identifies the build and duration of the 150,000-payment series.

On archived builds, redundant certification and evidence replicated by INSTALL sustain service under the specified failures. Nine runs at 200 payments/s complete 216,000 payments with one member delayed or paused for 30 s. Gateway-loss controls in which one member holds a complete copy replicated by INSTALL show backup submission 2.503–2.508 s after INSTALL. Conflict controls produce neither dual conflicting QCs nor repeated principal/fee effects, while partial approvals remain a capacity cost.

The multi-identity measurements in supplementary Section S2 report per-payment CPU and HTTP-payload costs; 2,048 identities share clients. Historical memory growth is reported separately from backlog. The same section gives an LND regtest reference, whose completion event, persistence, and workload concurrency differ from the Next measurements.

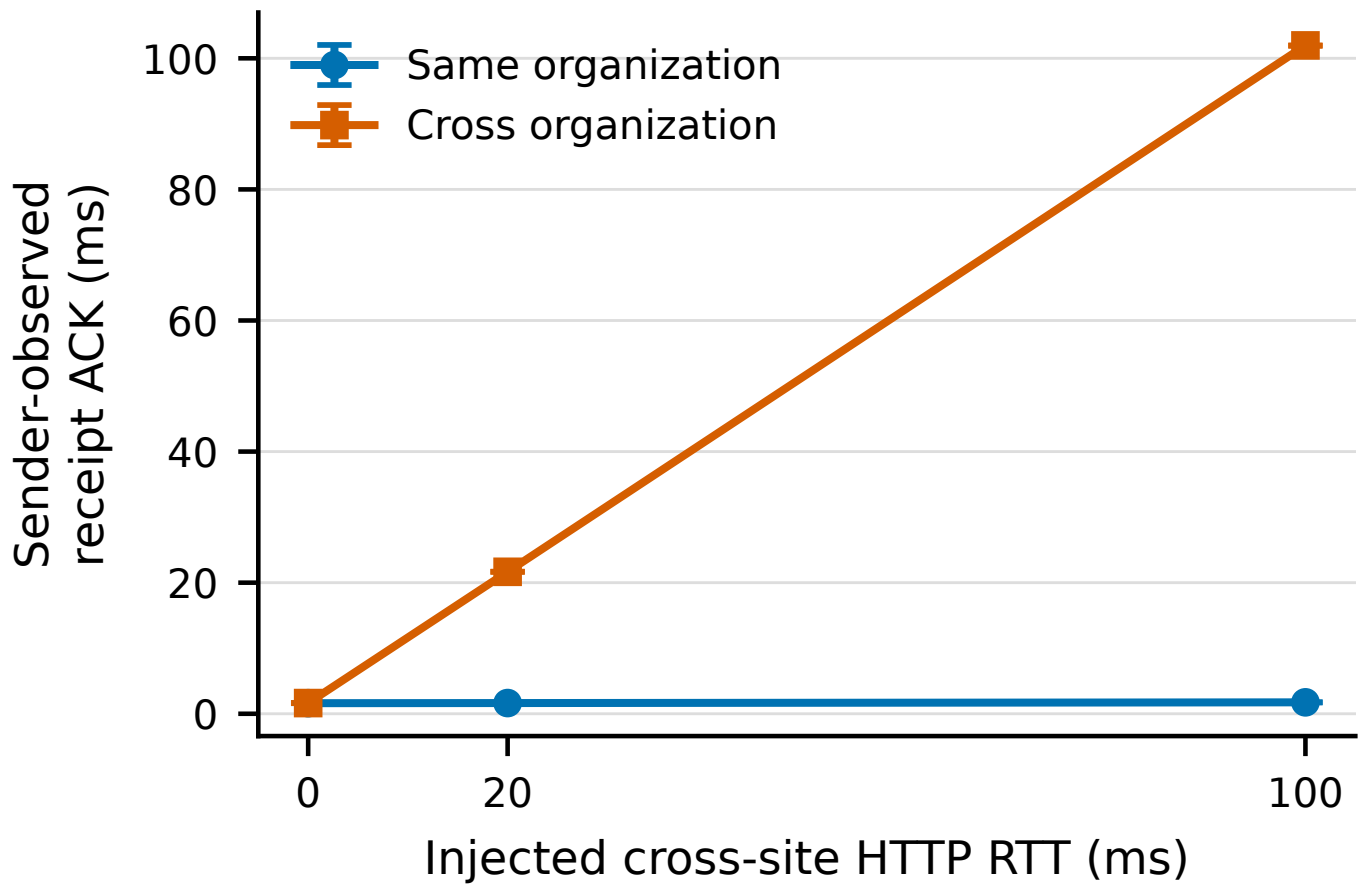


Fig. S6. ACK latency (ms) versus injected cross-site HTTP RTT (ms). Each point is the median of three run-level P50s for the first 120-s cohort; error bars span those P50s' minimum and maximum. Aggregate offered rate is 100 payments/s. Committee P2P is undelayed.

S5. PER-RUN EXPERIMENTAL RECORDS

TABLE S10

ARCHIVED ONE-ORGANIZATION OPERATING-POINT RUNS. EACH RUN SENDS AND COMPLETES 150,000 PAYMENTS. INJECTION AND COMPLETION ARE IN PAYMENTS/S; RECEIPT QUANTILES ARE IN MS. COMPLETION INCLUDES FINAL BACKGROUND DRAIN. PEAK IS THE MAXIMUM NUMBER OF PAYMENTS WITH UNFINISHED BACKGROUND PROCESSING; LIMIT HITS COUNTS TRIGGERS OF THE 4,096-UNFINISHED-PAYMENT LIMIT.

Run	Target	Injection	Completion	P50	P95	Peak	Limit hits
1	2000	1,964.53	1,948.66	2.521	50.499	2159	0
2	2000	1,969.65	1,950.45	2.566	50.761	2205	0
3	2200	2,127.70	2,103.63	4.538	69.463	2655	0
4	2200	2,127.15	2,106.29	4.553	69.945	2548	0
5	2400	2,206.95	2,174.68	7.174	89.376	4096	1635

TABLE S11

BASELINE SOURCE-RECOVERY RUNS. EACH OFFERS 1,200 TWO-HOP UNITS (2,400 PAYMENTS) OVER 60 SECONDS. RECEIPT QUANTILES FOR NORMAL CHILD PAYMENTS ARE IN MS; ELAPSED AND DEADLINE-TO-ALL-REPLICA-INSTALLATION OBSERVATION MEDIAN ARE IN SECONDS. THE LATTER INCLUDES 250-MS POLLING. EVERY RUN ENDS WITH A 60,000-CAL RESERVE, ZERO NET EXPENDITURE, AND ZERO GAP.

Seed	Withheld	Paid/recovered	Elapsed	Child P50	Child P95	Install P50
23	0%	0/0	61.540	1.251	1.620	–
23	1%	12/12	90.225	1.185	1.432	1.759
23	5%	60/60	90.096	1.191	1.536	1.257
37	0%	0/0	61.595	1.199	1.559	–
37	1%	12/12	91.935	1.190	1.479	1.509
37	5%	60/60	91.530	1.192	1.426	1.257
59	0%	0/0	61.550	1.191	1.461	–
59	1%	12/12	90.595	1.193	1.452	1.761
59	5%	60/60	90.346	1.194	1.494	1.507

TABLE S12

ARCHIVED FINITE-RESERVE RUNS. ONE 300-S RUN PER CONDITION; CAPITAL VALUES ARE CAL. CLOSED COUNTS REFER TO TWO-HOP UNITS. ADAPTIVE INCREASES ARE ACTUAL TRANSFERS FROM AN EXTERNAL FUNDING ACCOUNT. MEAN IS THE TIME-AVERAGE ORGANIZATION CAL BALANCE OBTAINED BY INTEGRATING BALANCE SAMPLES OVER THE 300-S INPUT WINDOW.

Condition	Initial	Added	Final	Mean	Closed
Constant / fixed low	3,600	0	3,600	3,600.0	46/6000
Constant / fixed high	14,400	0	14,400	14,400.0	6000/6000
Constant / adaptive	3,600	10,800	14,400	13,801.8	6000/6000
Step / fixed high	21,600	0	21,600	21,600.0	6000/6000
Step / adaptive	3,600	18,000	21,600	14,078.7	6000/6000
Delayed / fixed high	28,800	0	28,800	28,800.0	6000/6000
Delayed / adaptive	3,600	19,700	23,300	20,611.2	6000/6000

TABLE S13

SYNCHRONOUS-MEMBER CONTINUATION CONTROLS: 100 HOPS PER RUN, NINE DISK-BACKED SERVICES, USER-PAID FUEL, AND NoSYNC WALLETS. DURATIONS END AT THE LAST CERTIFIED RECEIPT AND ARE IN SECONDS. N/S ORDER APPLIES TO THE TWO FAST MODES; THE SYNCHRONOUS WAITING RUN FOLLOWS EACH PAIR.

Pair	Order	NoSync fast	Sync fast	Sync wait
1	N/S	0.681127	4.492812	64.969480
2	S/N	0.705604	4.309984	65.193522
3	N/S	0.636821	4.470844	65.012947
Median		0.681127	4.470844	65.012947